基于 Transformer 模型对以太坊价格预测

陈小龙(计算机应用技术)

2023 年 12 月 3 日

1. 创建模型

这个以太坊价格预测模型基于 Transformer 架构。它由编码器和解码器组成，每个部分都有多个层。编码器将输入数据进行编码，解码器则利用编码器的输出来生成预测结果。模型的核心是多头注意力机制和前馈神经网络层，它们允许模型对序列数据进行全局性的理解和学习。

```python
# 创建Transformer模型
class TransformerModel(nn.Module):
    def __init__(self, input_size, output_size, hidden_size=256, num_layers=6, num_heads=8, dropout=0.1):
        super(TransformerModel, self).__init__()
        self.embedding = nn.Linear(input_size, hidden_size)

        # 编码器层
        self.encoder_layer = nn.TransformerEncoderLayer(hidden_size, num_heads, hidden_size, dropout)
        self.transformer_encoder = nn.TransformerEncoder(self.encoder_layer, num_layers)

        # 解码器层
        self.decoder_layer = nn.TransformerDecoderLayer(hidden_size, num_heads, hidden_size, dropout)
        self.transformer_decoder = nn.TransformerDecoder(self.decoder_layer, num_layers)

        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        # 调整输入数据维度以匹配模型
        x = x.unsqueeze(-1)
        x = self.embedding(x)
        # 调整维度以适应Transformer的输入
        x = x.permute(1, 0, 2)

        # 编码器
        encoder_output = self.transformer_encoder(x)

        # 解码器
        decoder_output = self.transformer_decoder(x, encoder_output)
        # 恢复原始维度
        decoder_output = decoder_output.permute(1, 0, 2)
        # 仅使用最后一个时间步的输出
        output = self.fc(decoder_output[:, -1, :])
        return output
```

2. 数据处理和训练

数据处理的过程包括读取文件、转换为 Numpy 数组、归一化处理和转换为 PyTorch 张量。这确保了模型能够处理和训练以太坊价格数据。

```
# 读取数据
def get_data():
    file_path = "dataset.txt"
    with open(file_path, 'r') as file:
        file_content = file.read()
        lines = file_content.strip().split('\n')
        arrays = []
        for line in lines:
            values = line.split()
            float_values = [float(val) for val in values]
            arrays.append(np.array(float_values))
        big_array = np.array(arrays)
        return big_array.astype(np.float32)


# 模型和训练参数
input_size = 1
output_size = 1
model = TransformerModel(input_size, output_size)
criterion = nn.SmoothL1Loss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)


# 归一化数据
def normalize_data(data):
    return (data - np.mean(data)) / np.std(data)


# 处理数据
stock_data = get_data()
normalized_data = normalize_data(stock_data)
tensor_stock_data = torch.from_numpy(normalized_data)
```

3. 模型训练

模型在训练过程中使用了 Adam 优化器和 Smooth L1 Loss 损失函数进行迭代优化。通过迭代多个 Epoch，模型逐渐学习到数据的特征和模式。训练过程中，模型通过优化器进行参数更新，计算损失并进行反向传播。每个 Epoch 结束时，输出损失以监控模型的训练情况。

```
# 训练模型
num_epochs = 300
target_data = tensor_stock_data[:, 1:]
for epoch in range(num_epochs):
    optimizer.zero_grad()
    output = model(tensor_stock_data[:, :-1])
    loss = criterion(output, target_data)
    loss.backward()
    optimizer.step()
    if (epoch + 1) % 10 == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')
```

4. 模型预测

最终预测阶段，模型使用训练好的参数对最后一组数据的最后一个值进行预测。预测结果经过反归一化处理，将其转换回原始数据的尺度，并与实际值进行比较。这个比较展示了模型在预测以太坊价格方面的表现，以便评估模型的准确性和可靠性。

```
# 模型预测
with torch.no_grad():
    test_input = tensor_stock_data[-1, :-1].unsqueeze(0)
    predicted = model(test_input)
    predicted_denormalized = predicted.item() * np.std(stock_data) + np.mean(stock_data)
    actual_value = stock_data[-1, -1]
    print(f"Predicted: {predicted_denormalized}, Actual: {actual_value}")
```

5. 预测输出:

```
Epoch [10/300], Loss: 0.1408
Epoch [20/300], Loss: 0.1284
Epoch [30/300], Loss: 0.1121
Epoch [40/300], Loss: 0.1910
Epoch [50/300], Loss: 0.0655
Epoch [60/300], Loss: 0.0565
Epoch [70/300], Loss: 0.0430
Epoch [80/300], Loss: 0.0285
Epoch [90/300], Loss: 0.0210
Epoch [100/300], Loss: 0.0170
Epoch [110/300], Loss: 0.0195
Epoch [120/300], Loss: 0.0127
Epoch [130/300], Loss: 0.0133
Epoch [140/300], Loss: 0.0117
Epoch [150/300], Loss: 0.0151
Epoch [160/300], Loss: 0.0105
Epoch [170/300], Loss: 0.0114
Epoch [180/300], Loss: 0.0116
Epoch [190/300], Loss: 0.0141
Epoch [200/300], Loss: 0.0142
Epoch [210/300], Loss: 0.0127
Epoch [220/300], Loss: 0.0118
Epoch [230/300], Loss: 0.0106
Epoch [240/300], Loss: 0.0101
Epoch [250/300], Loss: 0.0118
Epoch [260/300], Loss: 0.0136
Epoch [270/300], Loss: 0.0108
Epoch [280/300], Loss: 0.0092
Epoch [290/300], Loss: 0.0103
Epoch [300/300], Loss: 0.0093
Predicted: 1910.5551554425983, Actual: 2063.389892578125
```

6. 总结:

该报告展示了一个基于 Transformer 架构的以太坊价格预测模型。预测结果为 1910.56，而实际值为 2063.39。预测值略微低于实际值，差异在 153 左右，这意味着模型在这个特定时间点上的预测略有偏差，可能需要更进一步的调整和改进以提高准确性。